

```

def matrix_chain_order(dims):
    """
    Matrix Chain Multiplication using DP
    dims: list of dimensions, matrix i has dims[i-1] x dims[i]
    Time: O(n^3), Space: O(n^2)
    """
    n = len(dims) - 1
    # m[i][j] = minimum multiplications for matrices i..j
    m = [[0] * (n + 1) for _ in range(n + 1)]
    s = [[0] * (n + 1) for _ in range(n + 1)]

    # l is the chain length
    for l in range(2, n + 1):
        for i in range(1, n - l + 2):
            j = i + l - 1
            m[i][j] = float('inf')
            for k in range(i, j):
                cost = m[i][k] + m[k+1][j] + dims[i-1] * dims[k] * dims[j]
                if cost < m[i][j]:
                    m[i][j] = cost
                    s[i][j] = k
    return m, s

def print_optimal_parens(s, i, j):
    if i == j:
        return f'A{i}'
    k = s[i][j]
    left = print_optimal_parens(s, i, k)
    right = print_optimal_parens(s, k + 1, j)
    return f'({left} x {right})'

def print_dp_table(m, n):
    print('\nDP Cost Table m[i][j]:')
    print(f'{"":>6}', end='')
    for j in range(1, n + 1):
        print(f'A{j:>8}', end='')
    print()
    for i in range(1, n + 1):
        print(f'A{i:<5}', end='')
        for j in range(1, n + 1):
            if j < i:
                print(f'{"---":>9}', end='')
            else:
                print(f'{m[i][j]:>9}', end='')
        print()

# A1(10x30), A2(30x5), A3(5x60), A4(60x10)
dims = [10, 30, 5, 60, 10]
n = len(dims) - 1

print(f'Matrix Dimensions:')
for i in range(n):
    print(f'  A{i+1}: {dims[i]} x {dims[i+1]}')

m, s = matrix_chain_order(dims)
print(f'\nMinimum scalar multiplications: {m[1][n]}')
print(f'Optimal parenthesization: {print_optimal_parens(s, 1, n)}')
print_dp_table(m, n)

```

Matrix Dimensions:

A1: 10 x 30
A2: 30 x 5
A3: 5 x 60
A4: 60 x 10

Minimum scalar multiplications: 5000

Optimal parenthesization: ((A1 x A2) x (A3 x A4))

DP Cost Table m[i][j]:

	A	1A	2A	3A	4
A1		0	1500	4500	5000
A2		---	0	9000	4500
A3		---	---	0	3000
A4		---	---	---	0

```

def is_safe(board, row, col):
    for prev_row in range(row):
        placed = board[ prev_row ]
        if placed == col:
            return False

```

```

        if abs(prev_row - row) == abs(placed - col):
            return False
        return True

def solve_n_queens(n):
    board = [ -1 ] * n
    solutions = list()
    backtrack_count = 0

    def backtrack(row):
        nonlocal backtrack_count
        if row == n:
            solutions.append(list(board))
            return
        for col in range(n):
            if is_safe(board, row, col):
                board[ row ] = col
                backtrack(row + 1)
                board[ row ] = -1
            backtrack_count += 1

    backtrack(0)
    return solutions, backtrack_count

def display_board(solution, n):
    print(' + ' + '---+' * n)
    for row in range(n):
        print(' |', end='')
        for col in range(n):
            if solution[ row ] == col:
                print(' Q |', end='')
            else:
                print(' . |', end='')
        print()
    print(' + ' + '---+' * n)

n_values = list((4, 6, 8))
for n in n_values:
    solutions, backtracks = solve_n_queens(n)
    print(f'N={n}: {len(solutions)} solutions, {backtracks} backtracks')
    if n == 4:
        print(f'\n All solutions for {n}-Queens:')
        for i, sol in enumerate(solutions, 1):
            print(f'\n Solution {i}: {sol}')
            display_board(sol, n)

```

N=4: 2 solutions, 15 backtracks

All solutions for 4-Queens:

Solution 1: [1, 3, 0, 2]

```

+---+---+---+
| . | Q | . | . |
+---+---+---+
| . | . | . | Q |
+---+---+---+
| Q | . | . | . |
+---+---+---+
| . | . | Q | . |
+---+---+---+

```

Solution 2: [2, 0, 3, 1]

```

+---+---+---+
| . | . | Q | . |
+---+---+---+
| Q | . | . | . |
+---+---+---+
| . | . | . | Q |
+---+---+---+
| . | Q | . | . |
+---+---+---+

```

N=6: 4 solutions, 149 backtracks
N=8: 92 solutions, 1965 backtracks

```

from itertools import permutations

INF = float('inf')

def tsp_brute_force(cost, n):
    cities = list(range(1, n))
    best_cost = INF
    best_path = None
    for perm in permutations(cities):

```

```

        path = list((0,)) + list(perm) + list((0,))
        c = sum(cost[ path[ i ] ][ path[ i + 1 ] ] for i in range(n))
        if c < best_cost:
            best_cost = c
            best_path = path
    return best_path, best_cost

cost = [
    [INF, 10, 8, 9, 7],
    [ 10, INF, 10, 5, 6],
    [ 8, 10, INF, 8, 9],
    [ 9, 5, 8, INF, 6],
    [ 7, 6, 9, 6, INF]
]
n = 5
cities = list(('A', 'B', 'C', 'D', 'E'))
best_path, best_cost = tsp_brute_force(cost, n)

print('5-City TSP - Cost Matrix:')
print(f'{"":>4}', ' '.join(f'{c:>5}' for c in cities))
for i, row in enumerate(cost):
    r = [ 'INF' if x == INF else str(x) for x in row ]
    print(f'{cities[ i ]:>4}', ' '.join(f'{v:>5}' for v in r))

print(f'\nOptimal Tour: " -> ".join(cities[ i ] for i in best_path)')
print(f'Minimum Cost: {best_cost}')
print(f'\nPath verification:')
for i in range(n):
    u, v = best_path[ i ], best_path[ i + 1 ]
    print(f' {cities[ u ]} -> {cities[ v ]}: cost = {cost[ u ][ v ]}')

```

5-City TSP - Cost Matrix:

	A	B	C	D	E
A	INF	10	8	9	7
B	10	INF	10	5	6
C	8	10	INF	8	9
D	9	5	8	INF	6
E	7	6	9	6	INF

Optimal Tour: A -> C -> D -> B -> E -> A

Minimum Cost: 34

Path verification:

A -> C: cost = 8
 C -> D: cost = 8
 D -> B: cost = 5
 B -> E: cost = 6
 E -> A: cost = 7

```

def first_fit(items, capacity=1.0):
    bins = list()
    bin_contents = list()
    for item in items:
        placed = False
        for i, space in enumerate(bins):
            if space >= item:
                bins[ i ] -= item
                bin_contents[ i ].append(item)
                placed = True
                break
        if not placed:
            bins.append(capacity - item)
            bin_contents.append(list((item,)))
    return bin_contents

def first_fit_decreasing(items, capacity=1.0):
    return first_fit(sorted(items, reverse=True), capacity)

def best_fit_decreasing(items, capacity=1.0):
    sorted_items = sorted(items, reverse=True)
    bins = list()
    bin_contents = list()
    for item in sorted_items:
        best_idx = -1
        best_space = float('inf')
        for i, space in enumerate(bins):
            if space >= item and space - item < best_space:
                best_space = space - item
                best_idx = i
        if best_idx >= 0:
            bins[ best_idx ] -= item
            bin_contents[ best_idx ].append(item)

```

```

        else:
            bins.append(capacity - item)
            bin_contents.append(list((item,)))
        return bin_contents

def display_bins(label, bins):
    print(f'\n{label}: {len(bins)} bins')
    for i, b in enumerate(bins, 1):
        used = sum(b)
        bar = '#' * int(used * 20)
        print(f'  Bin {i}: {[ round(x,1) for x in b ]} | Used: {used:.1f} [{bar:<20}]')

items = list((0.5, 0.7, 0.3, 0.9, 0.2, 0.6, 0.8, 0.4, 0.1, 0.5))
capacity = 1.0
lower_bound = -(-sum(items) // capacity)

print(f'Items: {items}')
print(f'Capacity: {capacity}')
print(f'Sum of items: {sum(items)}')
print(f'Lower bound on bins: {int(lower_bound)}')

ff_bins = first_fit(items)
ffd_bins = first_fit_decreasing(items)
bfd_bins = best_fit_decreasing(items)

display_bins('First Fit (FF)', ff_bins)
display_bins('First Fit Decreasing (FFD)', ffd_bins)
display_bins('Best Fit Decreasing (BFD)', bfd_bins)

print(f'\nSummary: Lower Bound={int(lower_bound)}, FF={len(ff_bins)}, FFD={len(ffd_bins)}, BFD={len(bfd_bins)}')

```

```

Items: [0.5, 0.7, 0.3, 0.9, 0.2, 0.6, 0.8, 0.4, 0.1, 0.5]
Capacity: 1.0
Sum of items: 5.0
Lower bound on bins: 5

```

```

First Fit (FF): 6 bins
  Bin 1: [0.5, 0.3, 0.2] | Used: 1.0 [#####]
  Bin 2: [0.7, 0.1] | Used: 0.8 [##### ]
  Bin 3: [0.9] | Used: 0.9 [##### ]
  Bin 4: [0.6, 0.4] | Used: 1.0 [#####]
  Bin 5: [0.8] | Used: 0.8 [##### ]
  Bin 6: [0.5] | Used: 0.5 [##### ]

```

```

First Fit Decreasing (FFD): 6 bins
  Bin 1: [0.9] | Used: 0.9 [##### ]
  Bin 2: [0.8, 0.1] | Used: 0.9 [##### ]
  Bin 3: [0.7, 0.3] | Used: 1.0 [#####]
  Bin 4: [0.6, 0.4] | Used: 1.0 [#####]
  Bin 5: [0.5, 0.5] | Used: 1.0 [#####]
  Bin 6: [0.2] | Used: 0.2 [#### ]

```

```

Best Fit Decreasing (BFD): 6 bins
  Bin 1: [0.9] | Used: 0.9 [##### ]
  Bin 2: [0.8, 0.1] | Used: 0.9 [##### ]
  Bin 3: [0.7, 0.3] | Used: 1.0 [#####]
  Bin 4: [0.6, 0.4] | Used: 1.0 [#####]
  Bin 5: [0.5, 0.5] | Used: 1.0 [#####]
  Bin 6: [0.2] | Used: 0.2 [#### ]

```

```

Summary: Lower Bound=5, FF=6, FFD=6, BFD=6

```

```

import random
import time
import sys

sys.setrecursionlimit(20000)
comparisons = 0

def partition(arr, low, high):
    global comparisons
    pivot = arr[ high ]
    i = low - 1
    for j in range(low, high):
        comparisons += 1
        if arr[ j ] <= pivot:
            i += 1
            arr[ i ], arr[ j ] = arr[ j ], arr[ i ]
    arr[ i + 1 ], arr[ high ] = arr[ high ], arr[ i + 1 ]
    return i + 1

def deterministic_quicksort(arr, low, high):
    if low < high:

```

```

    pi = partition(arr, low, high)
    deterministic_quicksort(arr, low, pi - 1)
    deterministic_quicksort(arr, pi + 1, high)

def randomized_quicksort(arr, low, high):
    if low < high:
        rand_idx = random.randint(low, high)
        arr[ rand_idx ], arr[ high ] = arr[ high ], arr[ rand_idx ]
        pi = partition(arr, low, high)
        randomized_quicksort(arr, low, pi - 1)
        randomized_quicksort(arr, pi + 1, high)

def run_test(sort_fn, arr):
    global comparisons
    a = list(arr)
    comparisons = 0
    start = time.perf_counter()
    sort_fn(a, 0, len(a) - 1)
    elapsed = (time.perf_counter() - start) * 1000
    return comparisons, elapsed

N = 1000
test_cases = {
    'Random'      : [ random.randint(1, 100000) for _ in range(N) ],
    'Sorted'      : list(range(N)),
    'Reverse'     : list(range(N, 0, -1)),
    'Nearly Sorted': list(range(N))
}

ns = test_cases[ 'Nearly Sorted' ]
for _ in range(N // 20):
    i, j = random.randint(0, N-1), random.randint(0, N-1)
    ns[ i ], ns[ j ] = ns[ j ], ns[ i ]

print(f'{"Input Type":<16} {"DQS Comps":>12} {"DQS Time(ms)":>14} {"RQS Comps":>12} {"RQS Time(ms)":>14}')
print('-' * 72)

for case, arr in test_cases.items():
    d_comps, d_time = run_test(deterministic_quicksort, arr)
    r_comps, r_time = run_test(randomized_quicksort, arr)
    print(f'{case:<16} {d_comps:>12} {d_time:>14.2f} {r_comps:>12} {r_time:>14.2f}')

```

Input Type	DQS Comps	DQS Time(ms)	RQS Comps	RQS Time(ms)
Random	10083	1.73	10581	2.04
Sorted	499500	72.84	10081	1.84
Reverse	499500	60.57	12046	2.16
Nearly Sorted	56282	6.83	10437	1.92